

BACKEND DEVELOPER PORTFOLIO

CONTACT

E. sowork02@gmail.com

GitHub. <https://github.com/wishs2>

01	ABOUT ME	자기소개
02	PROJECT	프로젝트 소개
03	PROBLEM SOLVING	문제 해결 경험
04	RETROSPECTIVE	프로젝트 회고 및 개선 방향
05	EDUCATION & CERTIFICATIONS	교육 및 자격

01 ABOUT ME

BACKEND DEVELOPER

기능이 동작하는 것에서 그치지 않고, 실제 데이터와 테스트를 통해 문제를 발견하고 구조를 개선합니다.

Java와 Spring Boot를 기반으로 리뷰·영수증 서비스를 개발하며
REST API, MySQL 기반 데이터 설계, 외부 API 연동을 경험했습니다.

프로젝트에서는 실제 영수증 데이터의 추출 오류를 분석하고,
데이터 무결성과 외부 API 호출 구조까지 개선하며 안정적인 처리 방식을 고민했습니다.

01 | 실제 데이터로 문제를 검증합니다.

OCR 응답의 위치·라인·신뢰도 정보를 활용하도록 처리 구조를 재설계하고,
동일 영수증 8건의 Before/After 테스트로 **필드 정확도 46.2% → 97.4%**를 검증했습니다.

02 | 데이터 규칙을 저장 단계까지 보장합니다.

영수증과 리뷰의 중복 기준을 **Composite UNIQUE Constraint**로 표현하여
애플리케이션 로직뿐 아니라 DB 저장 단계에서도 데이터 무결성을 보장했습니다.

03 | 외부 API의 호출 구조까지 설계합니다.

리뷰마다 LLM을 호출하는 대신 **Event → Redis Set → Scheduler** 구조로 변경하여
동일 업체의 변경 요청을 통합하고 변경된 업체만 요약하도록 구성했습니다.

02 PROJECT

위치 및 영수증 기반 뷰티샵 리뷰 서비스

>>> Project Overview

기간

- 2025.02 ~ 2025.03
- 이후 리팩토링 진행 ~ 2026.02

팀 구성

- 4인 팀 프로젝트

역할

- Backend Developer · Team Lead
- WBS 작성 및 일정 관리
- 업무 분배
- 코드 리뷰 및 Notion 협업

개요

- 영수증 OCR을 통해 결제 정보를 검증하고, 검증된 영수증을 기반으로 뷰티샵 리뷰를 작성할 수 있는 서비스



02 PROJECT

위치 및 영수증 기반 뷰티샵 리뷰 서비스

>>> Project Overview

기술 스택

- Backend
Java 17 · Spring Boot 3 · MyBatis · Spring Security
- Database / Cache
MySQL 8 · Redis
- Frontend
JavaScript · Thymeleaf · Axios
- External API
Naver CLOVA OCR · OpenAI API
- Testing
JUnit · Mockito · Postman · Swagger · JMeter
- Infrastructure
AWS EC2 · AWS RDS · AWS S3 · Docker
※ AWS 인프라 구성 및 배포는 팀원이 담당
- Collaboration
Git · GitHub · Notion · Figma

주요 기능

- 회원 및 인증
 - 회원가입 · 로그인 · 소셜 로그인
 - 이메일/SMS 인증 및 계정 관리
- 뷰티샵 탐색
 - 위치 기반 뷰티샵 조회
 - 뷰티샵 상세 정보 및 사업자 정보 조회
 - 주소 검색 · 지도 기반 위치 조회
- 영수증 인증 및 리뷰
 - OCR 기반 영수증 결제 정보 인증
 - 영수증 기반 리뷰 작성 · 조회
 - LLM 기반 리뷰 요약
- 마이페이지 / 관리자
 - 나의 리뷰 및 회원정보 관리
 - 뷰티샵 등록 · 수정 · 삭제
 - 회원 · 뷰티샵 관리

02 PROJECT

위치 및 영수증 기반 뷰티샵 리뷰 서비스

>>> Project Overview

기여 부분

Backend

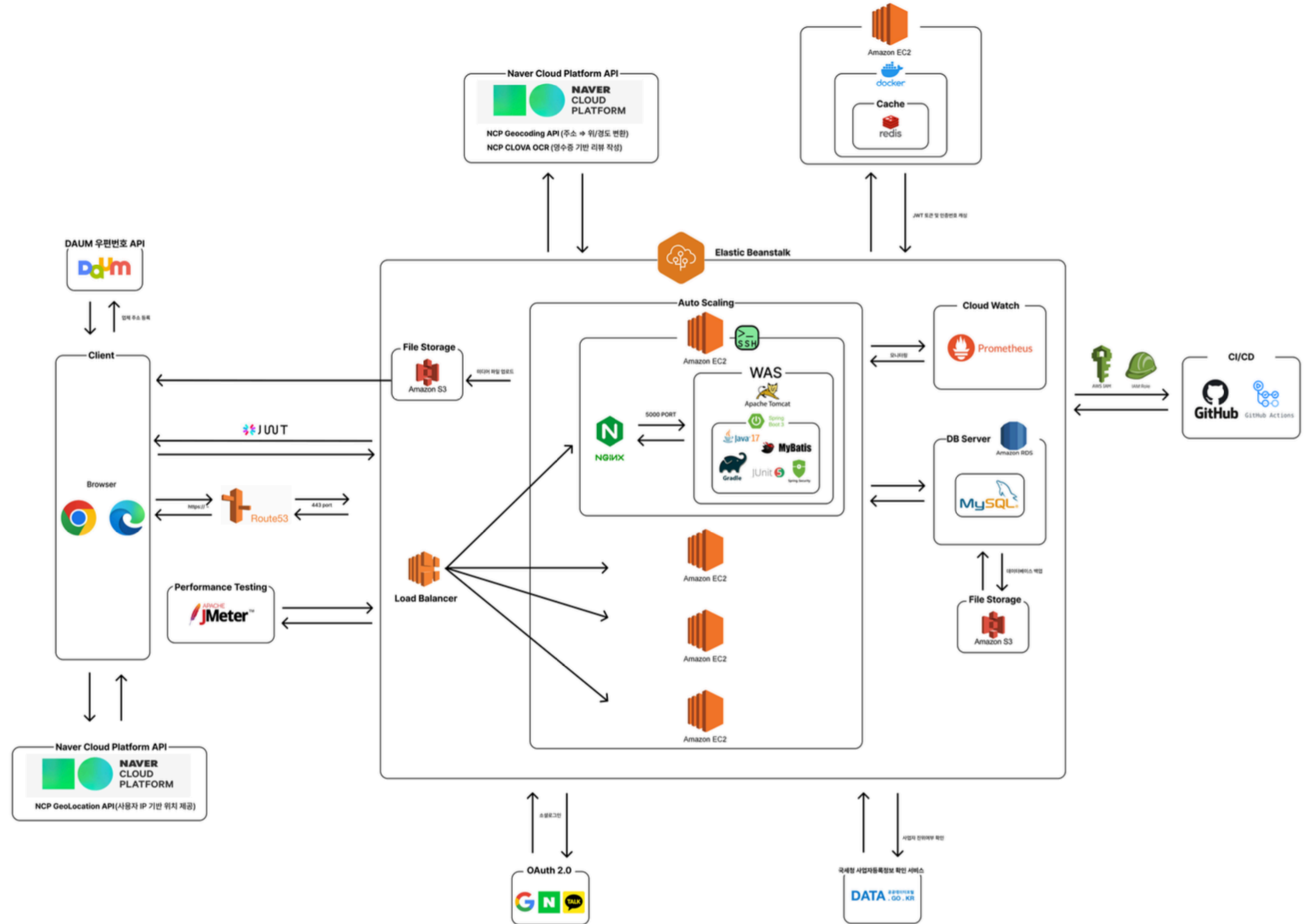
- 영수증 OCR
 - Naver CLOVA OCR API 연동
 - 영수증 주요 정보 추출 로직 구현
 - OCR 데이터 파싱 구조 개선
- 리뷰
 - 리뷰 등록 · 조회 · 수정 · 삭제 구현
 - 리뷰 이미지 및 키워드 처리
 - 회원/관리자별 접근 권한 검증
- 데이터베이스
 - 영수증 및 리뷰 테이블 설계
 - 중복 영수증 및 중복 리뷰 방지를 위한 UNIQUE 제약조건 적용
- LLM
 - 리뷰 생성 이벤트 기반 요약 대상 수집
 - Redis Set을 활용한 중복 대상 제거
 - Scheduler 기반 리뷰 요약 갱신

Team Lead / UI

- 협업 및 프로젝트 관리
 - WBS 작성 및 일정 관리
 - 업무 분배 및 진행 상황 관리
 - GitHub Branch · PR · Code Review 기반 협업
- UI
 - Header / Footer 및 주요 화면 Layout 설계·구현

ARCHITECTURE

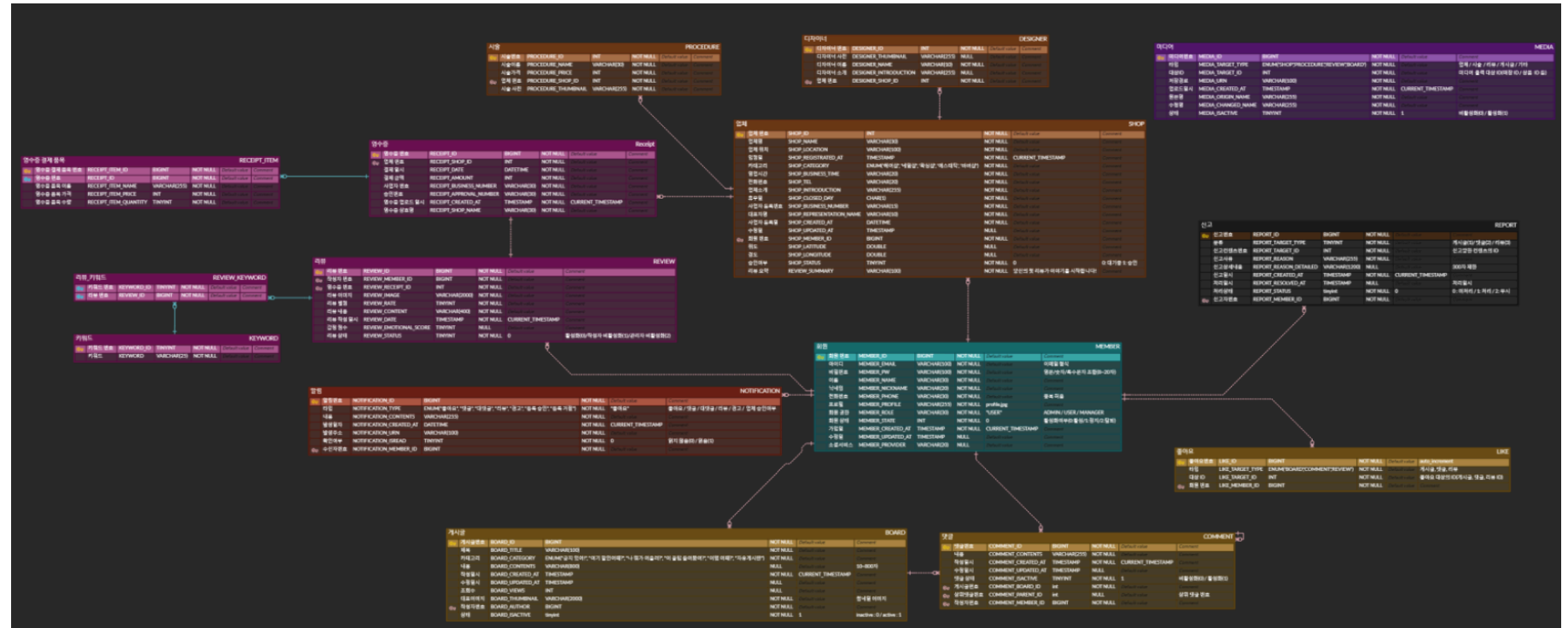
>>> 프로젝트 구조



ERD

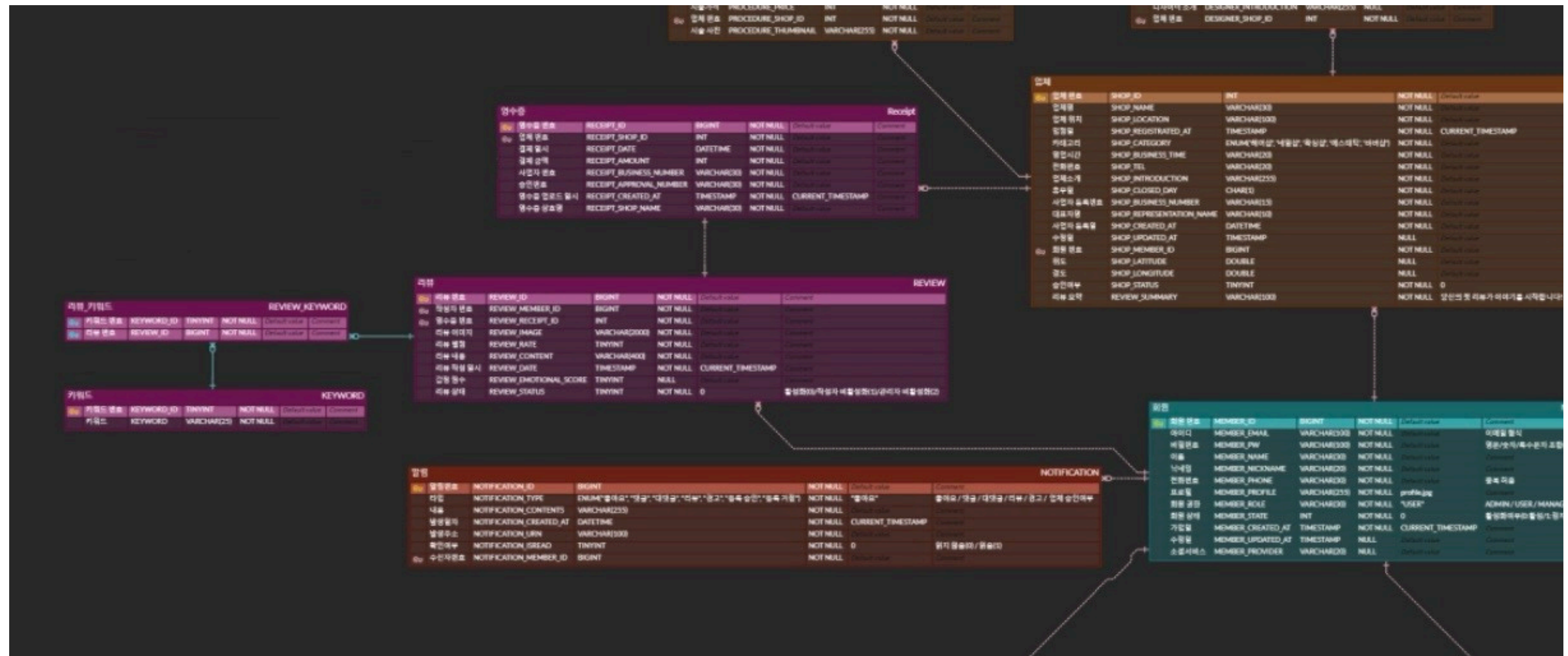
>>> Database Design

전체 ERD



담당 ERD

- Receipt
- Review
- Review
- Review_Keyword
- Keyword



03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

01. 문제 정의 — 구조 정보가 소실되는 String + Regex 처리

OCR 응답을 하나의 문자열로 변환해 Regex로 탐색하던 구조에서, 영수증 형식에 따라 동일한 필드의 위치와 표현이 달라지며 추출 정확도가 떨어지는 문제를 확인했습니다.

문제 상황

- 초기 구현에서는 빠른 기능 구현을 위해 OCR 응답 전체를 String으로 변환한 뒤 Regex로 필요한 필드를 탐색했습니다.

문제 발생

영수증 형식에 따른 추출 편차

- 동일 필드도 영수증마다 위치·표현 방식이 달라 Regex만으로 대응하기 어려움
- String 변환 과정에서 Token의 위치·Line·Confidence 정보 소실
- 예외 대응 규칙이 하나의 로직에 누적되어 수정 영향 범위 증가



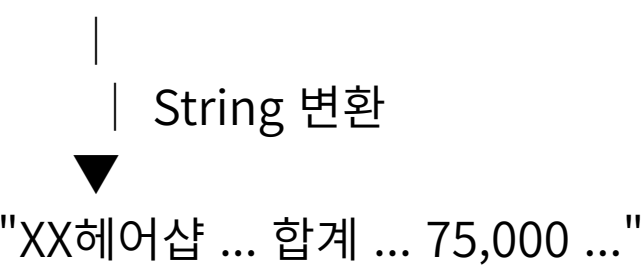
원인 파악

문제의 핵심은 Regex 자체보다, OCR 응답을 단일 문자열로 평탄화하면서 필드 판단에 활용할 수 있는 위치·라인·신뢰도 정보를 잃는 처리 구조에 있었습니다.

핵심 문제

OCR JSON

Token A — 위치
Token B — 위치
Token C — Confidence
Token D — 위치



↓
위치 정보 ×
라인 관계 ×
Confidence ×

↓
Regex가 문자열 표현에 의존

OCR 데이터는 구조화되어 있었지만 추출 과정에서는 비구조화 문자열처럼 처리하고 있었습니다.

03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

02. 구조 재설계 — Token → Line → Field Pipeline

구조화된 OCR 처리 파이프라인으로 재설계

전체 문자열을 직접 탐색하는 방식에서 벗어나
OCRToken → OCRLine → Field Extractor 단계로 처리 구조를 분리했습니다.

OCR의 위치·라인·Confidence 정보를 유지한 채 각 필드가 필요한 문맥을
독립적으로 판단하도록 설계했습니다.

① 구조 정보 보존

OCR JSON → OCRToken → OCRLine

- Text뿐 아니라 위치·Line·Confidence를 보존하여 추출 과정에서 활용

② 필드별 책임 분리

ReceiptFieldExtractor
├ ShopNameExtractor
├ AmountExtractor
├ DateTimeExtractor
├ BusinessNumberExtractor
└ ApprovalNumberExtractor

- 필드별 규칙을 독립시켜 변경 영향 범위를 제한

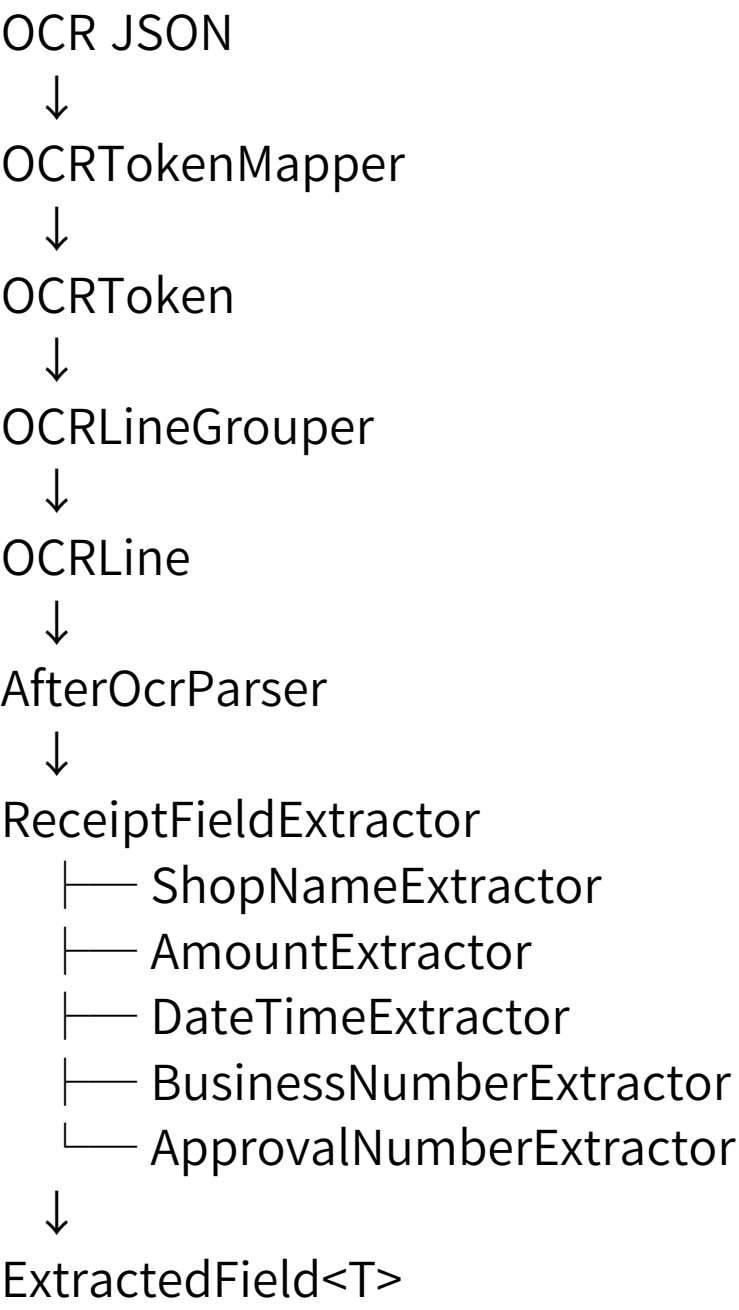
③ 후보 기반 판단

Pattern + Keyword + Context + Confidence

- 단일 Regex 일치가 아닌 후보 평가 방식으로 전환

- 각 후보에 Pattern·Keyword·주변 Line·Confidence를
조합하여 점수를 부여하고 최종 후보를 선택했습니다.

개선 후 구조



03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

03. 추출 전략 — Pattern에서 Context 기반 후보 평가로

문자열 매칭에서 문맥 기반 필드 추출로

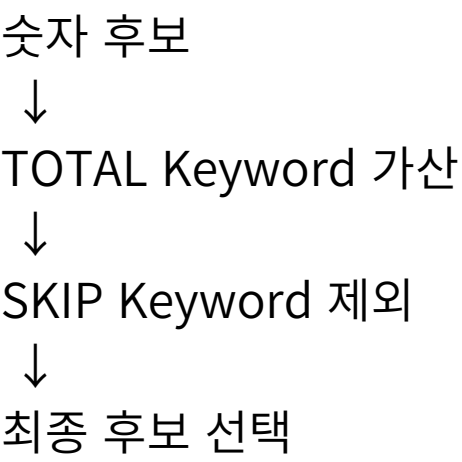
영수증마다 필드의 위치와 표현이 달라지는 문제에 대응하기 위해, 각 Extractor가 **패턴·키워드·주변 Line·Confidence**를 조합하여 후보를 평가하도록 개선했습니다.

공통 결과 모델 — ExtractedField

Value + Confidence + SourceText

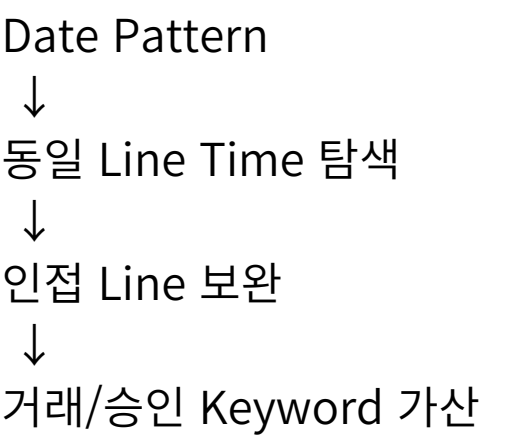
- 추출값과 판단 근거를 함께 보존하여 검증 및 실패 분석에 활용

① 결제금액



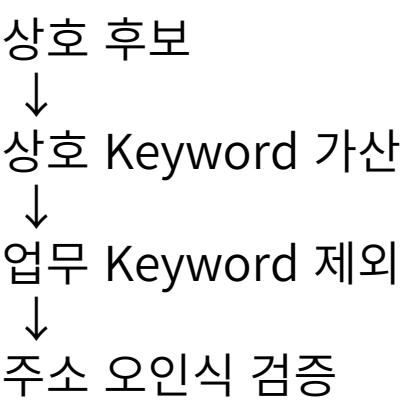
- 숫자 패턴 자체가 아니라 **금액 주변의 의미를 함께 평가**

② 결제일시



- 날짜와 시간이 서로 다른 OCR Line으로 분리되는 경우까지 고려하여 인접 문맥을 탐색

③ 상호명



- 짧은 한글 상호가 주소 패턴의 일부로 오인되어 제외되는 케이스를 확인하고, 주소 판별 조건과 상호 후보 검증 규칙의 경계를 조정

03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

04. 정확도 검증 — 동일 데이터셋 Before / After Test

Before / After 테스트를 통한 추출 정확도 검증

동일한 8개 영수증 OCR 응답에 기존 로직과 개선 로직을 각각 적용하고, Ground Truth와 비교하여 필드별 추출 정확도를 측정했습니다.

① 테스트 설계

Dataset

영수증 8건

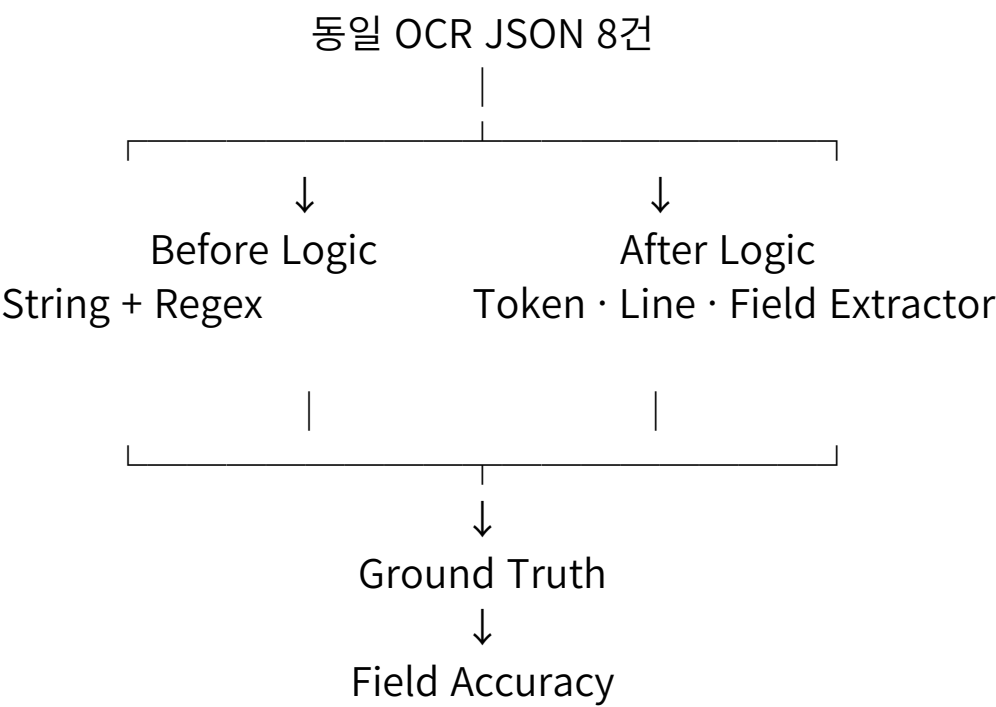
Evaluation Fields

상호명 · 결제금액 · 결제일시 · 사업자번호 · 승인번호

Total Evaluations

39 Cases

Evaluation



※ 사업자번호 1건은 Ground Truth 평가 대상에서 제외하여 총 39개 필드를 비교

② Overall Accuracy

46.2% → 97.4%

+51.2%p

③ Field Accuracy

Field	Before	After	변화
상호명	25.0%	87.5%	+62.5%p
결제금액	12.5%	100.0%	+87.5%p
결제일시	100.0%	100.0%	—
사업자번호	71.4%	100.0%	+28.6%p
승인번호	25.0%	100.0%	+75.0%p
TOTAL	46.2%	97.4%	+51.2%p

03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

05. 결과 분석 — 개선 효과와 Remaining Case

④ Result Analysis

01. 결제금액 | 12.5% → 100%

Before
숫자 Pattern만으로 후보를 탐색하여 공급가액·사업자번호 등 다른 숫자를 오인식

After
TOTAL/SKIP Keyword + 주변 Line 문맥을 후보 평가에 반영

Result
8건 모두 결제금액을 정확히 추출

02. 숫자 필드 분리 | 승인 25→100 / 사업자 71.4→100

Before
여러 숫자 필드가 하나의 처리 로직에서 경쟁

After
BusinessNumber / ApprovalNumber Extractor로 책임 분리

Result
각 숫자의 의미를 독립적으로 판별하여 오인식 제거

03. 상호명 | 25→87.5

Before
승인금액, 영수증, 주소 일부 등이 상호 후보로 선택

After
상호 Keyword + 제외 Keyword + Line 문맥 적용

Result
8건 중 7건 정확 추출

⑤ Remaining Case

OCR 원문 인식 오류

Ground Truth
CJ올리브영(주)OO점

↓ OCR

CJ올리브영(주) XX광역시 YY구 OO점 xx로 123 ...

↓ Extractor

CJ올리브영(주)

- 최종 1건은 OCR 단계에서 상호명 일부가 주소와 혼합되고 점이 다른 문자로 인식되어 Extractor만으로 원래 상호명을 완전히 복원하기 어려웠습니다.
- 후처리 규칙을 계속 추가하기보다 OCR Token 좌표 기반 영역 분리 또는 매장 데이터와의 검증을 추가 개선 방향으로 판단했습니다.

결과

동일한 8개 영수증의 39개 평가 필드에서 추출 정확도를 **46.2% → 97.4% (+51.2%p)**로 개선했습니다.

남은 오류는 후처리 규칙을 추가하기보다 OCR 입력 단계의 구조적 한계로 분리하여 후속 개선 방향을 정의했습니다.

03 PROBLEM SOLVING

>>> ② 영수증/리뷰 데이터 무결성

01. 중복 범위 정의 — 데이터 무결성 규칙 설계

문제

① 동일 영수증 중복 저장

동일한 실물 영수증이 반복 제출될 경우 동일 데이터가 중복 저장될 수 있었습니다.

② 동일 영수증 기반 리뷰 중복 작성

하나의 회원이 동일한 영수증을 이용해 리뷰를 반복 작성할 수 있었습니다.

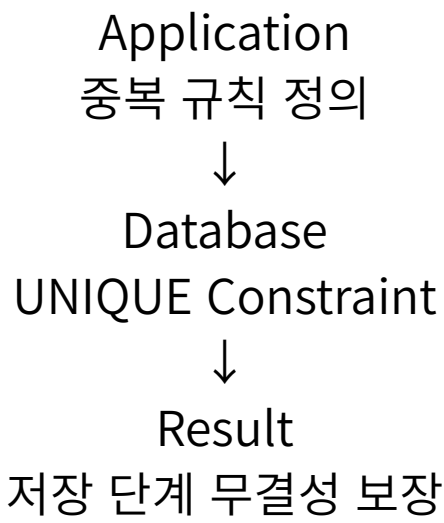
요구사항

- 영수증

리뷰
- 식별정보 동일 → 중복 저장 방지

동일 회원 + 동일 영수증 → 중복 리뷰 방지

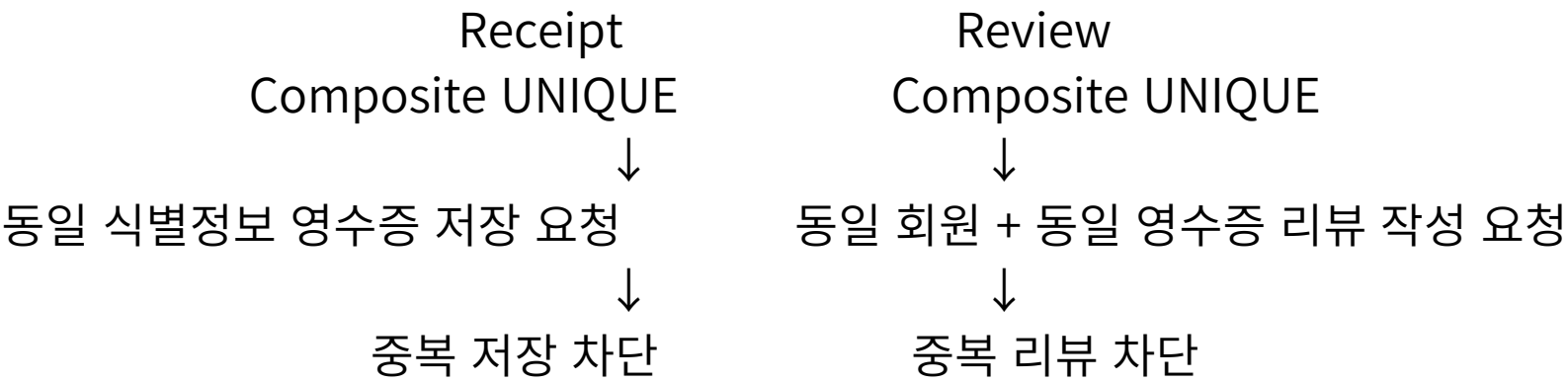
해결 방향



무결성 규칙

대상	식별 기준	DB 제약
Receipt	사업자번호 + 승인번호 + 결제일시 + 결제금액 + 매장명	Composite UNIQUE
Review	회원 ID + 영수증 ID	Composite UNIQUE

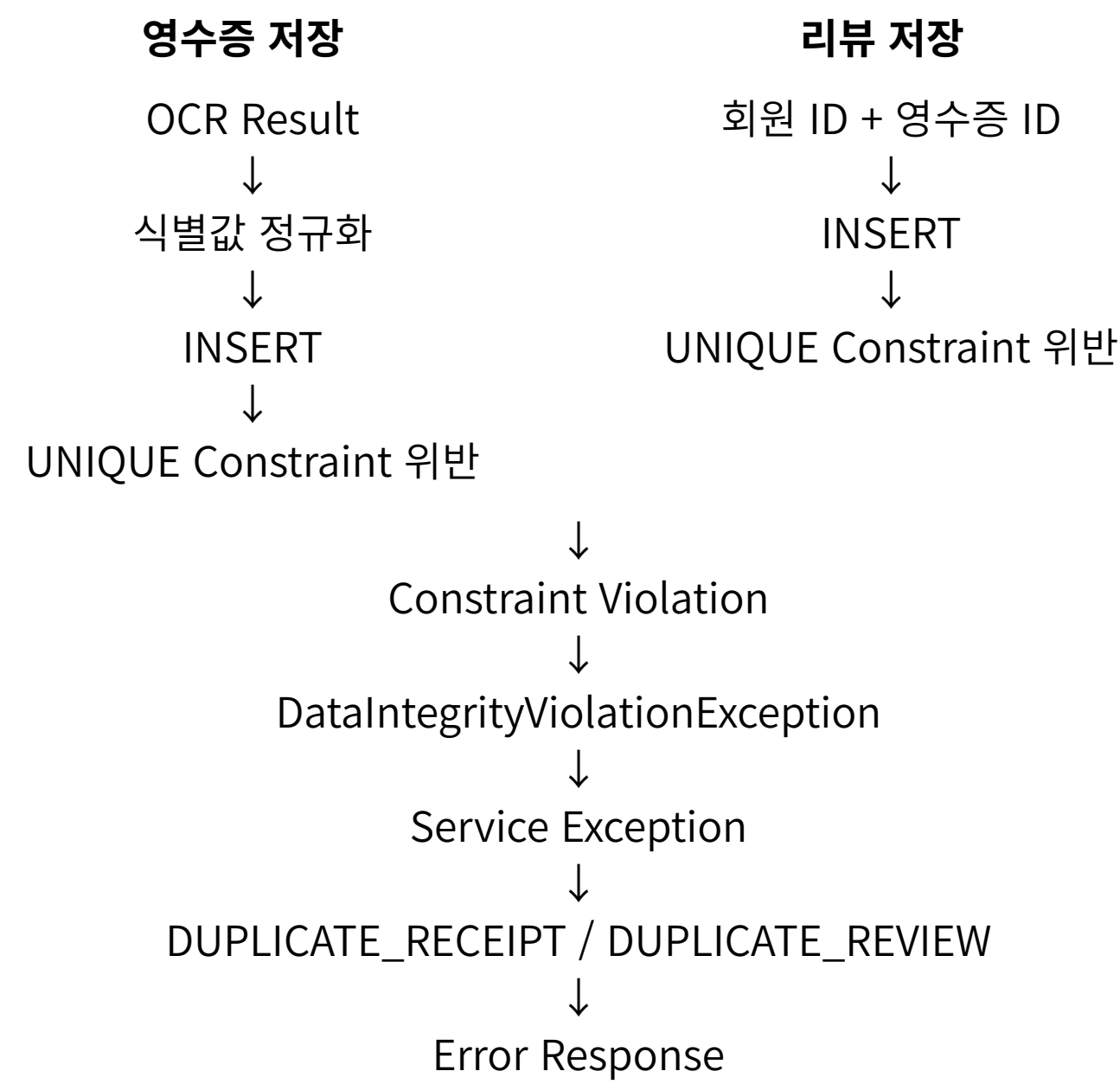
DB 제약 적용 결과



03 PROBLEM SOLVING

>>> ② 영수증/리뷰 데이터 무결성

02. 무결성 보장 — DB Constraint와 예외 처리



예외 처리

DB Constraint → DataIntegrityViolationException → Service Exception → Error Response

DB의 제약조건 위반을 서비스 계층의 의미 있는 예외로 변환하여 중복 요청에 대한 명확한 오류 응답을 제공

결과

중복 판별 규칙을 서비스 코드에만 두지 않고 DB Constraint로 명시하여 데이터 저장 단계에서 무결성을 보장했습니다.

DATA INTEGRITY

DB Constraint 기반
중복 데이터 저장 방지

ERROR HANDLING

DB 예외 → 서비스 예외
명확한 오류 응답

RESPONSIBILITY

Application — 업무 규칙 처리
Database — 최종 무결성 보장

03 PROBLEM SOLVING

>>> ③ LLM 리뷰 요약 호출 최적화

01. 반복 호출 문제 — 호출 단위 재설계

Review A → LLM → Summary Update
Review B → LLM → Summary Update
Review C → LLM → Summary Update
⋮

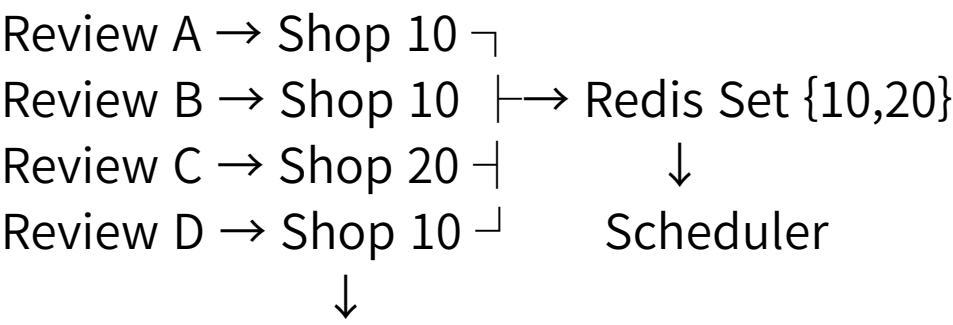
동일 업체에 대한 리뷰가 연속해서 작성될 경우 반복적인 요약 요청 발생

문제

리뷰 작성 이벤트마다 요약을 갱신하여, 동일 업체에 여러 리뷰가 연속 등록될 경우 LLM API 호출이 반복되는 구조였습니다.

해결 방향

리뷰 작성 이벤트에서 즉시 LLM을 호출하지 않고, 변경된 업체 ID를 Redis Set에 수집한 뒤 일정 주기로 일괄 처리하도록 변경했습니다.



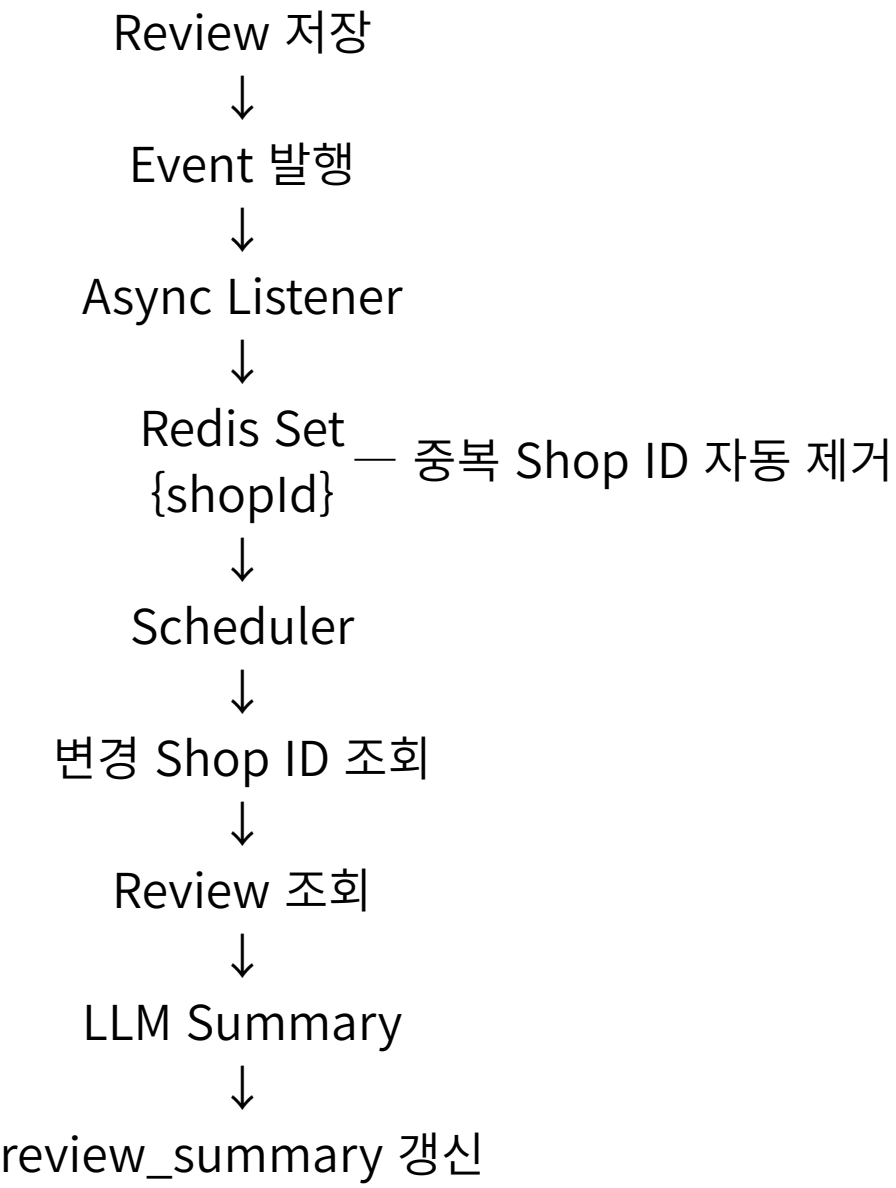
업체 단위 처리 — 리뷰 단위 처리에서 변경 업체 단위 처리로 전환

Redis Set 선택 이유

업체 ID의 중복을 자동 제거하여 동일 업체의 여러 변경 이벤트를 하나의 처리 대상으로 통합

02. Event → Redis → Scheduler 기반 처리 구조

전체 처리 흐름



03 PROBLEM SOLVING

>>> ③ LLM 리뷰 요약 호출 최적화

역할 분리

① EVENT

리뷰 변경 사실 전달

② ASYNC LISTENER

요약 대상 수집을 리뷰 저장 흐름에서 분리

③ REDIS SET

Shop ID 저장 + 중복 제거

④ SCHEDULER

수집된 업체의 요약 작업 실행

⑤ LLM SERVICE

리뷰 기반 요약 생성

- FAILURE HANDLING

업체 단위 예외 처리로 하나의 요약 실패가
이후 업체 처리까지 중단시키지 않도록 구성

03. 개선 결과와 Trade-off

구분	Before	After
처리 기준	리뷰 이벤트	변경 업체
대상 관리	개별 이벤트 처리	Redis Set 으로 변경 업체 수집
동일 업체 변경	반복 처리 가능	Shop ID 통합
처리 시점	이벤트 발생 시	Scheduler 주기 처리
처리 범위	이벤트마다 요약	변경 업체만 요약

결과

DEDUPLICATION

동일 업체의 여러 변경을 하나의 처리 대상으로 통합

DEFERRED PROCESSING

리뷰 작성과 LLM 요약 처리 시점 분리

TARGETED UPDATE

변경된 업체만 조회하여 요약 갱신

Trade-off

호출 효율 ↔ 요약 최신성

변경 요청을 모아 주기적으로 처리하는 대신 리뷰 작성과 요약 갱신 사이에
지연이 발생할 수 있습니다.

Further Improvement

실제 운영 환경에서는 리뷰 발생량과 요약 최신성 요구를 기준으로 Scheduler 주기를
조정하고, 실패한 요약 작업의 재처리 전략을 추가할 수 있습니다.

03 PROBLEM SOLVING

>>> ④ 리뷰 조회 API 부하 테스트 및 성능 검증

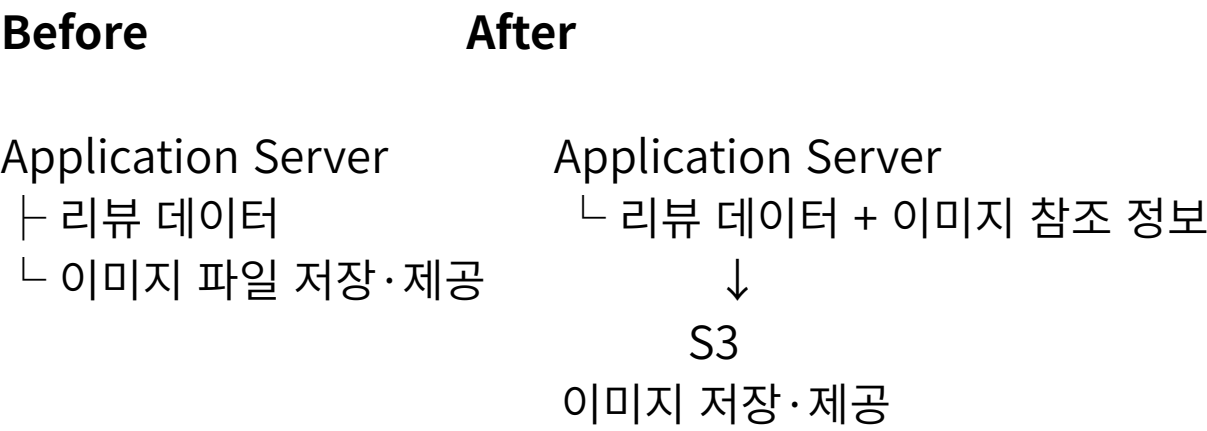
문제

이미지 파일의 저장·제공 책임이 애플리케이션 서버에 함께 존재

- Application Server
 - └ 리뷰 데이터 처리
 - └ 이미지 파일 저장·제공

해결 방향

이미지 저장·제공 책임을 S3로 분리하고, 구조 변경 전후 리뷰 조회 API의 부하 테스트 결과를 비교했습니다.



03 PROBLEM SOLVING

>>> ⑤ Testing & Collaboration

Testing

1. API Test

- Postman / Swagger
- REST API 호출 및 주요 CRUD 기능 검증

2. Unit Test

- JUnit / Mockito
- 비즈니스 로직 중심의 단위 테스트 작성

3. Performance Test

- JMeter · 리뷰 조회 API 30,000건 부하 테스트

WBS

1. 주제선정 및 피드백	2025-02-11	2025-02-17	100%		
1.1 아이디어 산출 및 구체화	2025-02-11	2025-02-14	완료	▼	공통 ▼
1.2 피드백	2025-02-17	2025-02-17	완료	▼	공통 ▼
1.3 피드백 내용 보완	2025-02-17	2025-02-18	완료	▼	공통 ▼
2. 분석 / 설계	2025-02-11	2025-02-17	100%		
2.1 역할 구성	2025-02-11	2025-02-11	완료	▼	공통 ▼
2.2 요구사항정의	2025-02-11	2025-02-11	완료		
2.3 문서 작성	2025-02-12	2025-02-18	완료		
2.3.1 기획서 작성	2025-02-11	2025-02-11	완료		
2.3.2 요구사항 정의서 작성	2025-02-11	2025-02-11	완료		
2.3.3 WBS 작성	2025-02-11	2025-02-11	완료		
2.3.4 FLOW CHART 작성	2025-02-12	2025-02-14	완료		
2.3.5 애플리케이션 아키텍처 설계	2025-02-12	2025-02-12	완료		
2.3.6 UseCase 작성			보류		
2.3.7 프로젝트 정의서	2025-02-11	2025-02-14	완료		
2.3.8 API 정의서 작성	2025-02-14	2025-02-16	완료		
2.3.9 수행일지 작성					
2.4 DB설계	2025-02-12	2025-02-13	완료		
2.5 와이어프레임 작성	2025-02-11	2025-02-19	완료		
2.6 UI/UX 설계	2025-02-11	2025-02-19	완료		

GitHub Issues

✓ [DESIGN] 리뷰 UI 구현 enhancement	1	MSW_002
Feature #118 · by wishs2 was closed on Mar 7, 2025		MSW_003
		MSW_004
✓ [FEAT] 마이페이지 리뷰 조회 기능 구현 enhancement	1	MSW_005
Feature #106 · by wishs2 was closed on Mar 5, 2025		MSW_006
		MSW_007
✓ [FEAT] 리뷰 삭제 기능 구현 enhancement	1	
Feature #97 · by wishs2 was closed on Mar 4, 2025		
✓ [FEAT] 리뷰 수정 기능 구현 enhancement	1	
Feature #80 · by wishs2 was closed on Feb 27, 2025		
✓ [FEAT] 리뷰 작성 기능 구현 enhancement	1	
Feature #70 · by wishs2 was closed on Feb 25, 2025		
✓ [FEAT] OCR 기능 구현 enhancement	1 2	
Feature #51 · by wishs2 was closed on Feb 24, 2025		

Collaboration

1. WBS & Schedule

- WBS 기반 기능별 업무 배정 및 일정·진행 상황 관리

2. GitHub Collaboration

- Branch · Issue · PR · Code Review를 통한 변경사항 관리

3. Documentation

- 요구사항 · API 명세 · 작업 범위 문서화

4. Team Lead

- 기능별 작업 조율 및 프로젝트 진행·발표 관리

Requirements

필터링	장당 30MB 이내
리뷰 유효기간	금칙어 필터링
별점	영수증에서 추출한 데이터 중 결제 일시를 활용하여 방문 후 30일 이내로만 리뷰 작성 가능
키워드	업체에 별점 🌟 매기기
영수증 리뷰 ***	작성자가 키워드 리뷰 작성 시 키워드 선택 이후 리뷰 조회 시 키워드를 통해 검색
리뷰 유효성 검증	결제 금액 검증, 결제 일자 검증, 상호명 영수증 증명 -> 이미 사용한 영수증인지 검증 DB 테이블에 저장

RETROSPECTIVE

프로젝트 회고 및 개선 방향

프로젝트를 통해 배운 점

01. 설계

빠른 구현보다 변경 가능한 구조를 고려하게 되었습니다.

OCR 데이터를 구조화하고 역할별 책임을 분리하면서 유지보수성을 고려한 설계의 중요성을 경험했습니다.

02. 데이터

데이터 무결성은 저장 단계부터 설계해야 한다는 것을 배웠습니다.

비즈니스 규칙을 DB 제약조건으로 구현하며 데이터 자체의 신뢰성을 확보하는 방법을 경험했습니다.

03. 외부 API

외부 API는 호출 횟수와 처리 시점까지 고려해야 한다는 것을 배웠습니다.

LLM 호출을 이벤트와 Scheduler 기반으로 분리하여 불필요한 반복 처리를 줄이는 구조를 경험했습니다.

04. 검증

구현 결과를 테스트하고 측정 조건까지 기록하는 중요성을 배웠습니다.

API·부하 테스트를 통해 기능과 성능을 확인하고, 재현 가능한 테스트 조건의 중요성도 경험했습니다.

향후 개선 방향

영역	현재 구현	향후 개선
OCR	Token → Line → Field 구조화	추출 규칙·신뢰도 기준 정교화 및 다양한 데이터 검증
Data Integrity	UNIQUE 제약조건 기반 중복 방지	OCR 변동성을 고려한 식별 기준 정교화, 예외 세분화
LLM	Event → Redis Set → Scheduler	실패 재처리, 입력량 관리, 처리 주기 개선
운영	개발·테스트 중심 경험	직접 배포, CI/CD 및 모니터링 경험 확장

앞으로의 개발 방향

이번 프로젝트를 통해 기능 구현뿐 아니라 데이터 처리 구조, 무결성, 외부 API 호출, 테스트까지 백엔드 개발 과정에서 발생하는 문제를 직접 개선해보았습니다.

앞으로는 재현 가능한 테스트와 운영 환경까지 범위를 확장하여, 구현 이후의 안정성과 유지보수성까지 고려하는 백엔드 개발자로 성장하고자 합니다.

04 EDUCATION & CERTIFICATIONS

EDUCATION & TRAINING

신안산대학교

- 2021년 3월 - 2023년 2월
- 호텔경영과 (4.28/4.5)

신세계아이앤씨

- 2024년 9월 - 2025년 3월
 - JAVA 기반 백엔드 개발자 양성과정
- 미니 팀 프로젝트 및 최종 프로젝트 수행

CERTIFICATIONS

정보처리산업기사

- 2025년 6월 취득

SQLD

- 2026년 6월 취득

THANK YOU